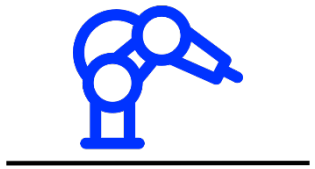
MAE 263F: Lecture 1

Mechanics of flexible structures and soft robots

Lecture 1: Agenda

- Newton's Method for Single Equation – Module 3
- Newton's Method for a System of Equations – Module 4
- Mass-Spring-Damper System (single DOF) – Module 5
- Implicit vs. Explicit Method – Module 6
- Mass-Spring-Damper System (multiple DOFs) – Module 7



Module 3

Newton’s method for nonlinear systems – single equation

- Algorithm
- Programming Implementation

Newton's Method

Assume that we have to solve a scalar equation

$$f(x) = 0, \tag{1.1}$$

whose solution is x^*.

Newton's Method

Assume that we have to solve a scalar equation

$$f(x) = 0, \tag{1.1}$$

whose solution is x^*. In Newton's method (also known as Newton-Raphson's method), we start with a guess x_0 and keep updating our approximation to the solution using the following rule:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \tag{1.2}$$

where $f'(x_n) = \frac{\partial f}{\partial x}|_{x_n}$ is the derivative of the function $f(x)$ with respect to x evaluated at $x = x_n$. After reaching the desired tolerance such that $|f(x_n)| < \epsilon$ where ϵ is a small number, we set the solution $x^* = x_n$.

Programming Implementation

As shown below, we can formulate a simple algorithm that implements Newton's method where the functions `f` and `df` evaluate $f(x)$ and $f'(x)$.

while $\texttt{err} > \epsilon$

$$\Delta x = \texttt{f}(x)/\texttt{df}(x)$$

$$x = x - \Delta x$$

$$\texttt{err} = \texttt{abs}(\texttt{f}(x))$$

end of while

Programming Implementation: exercise

- Let us solve the following equation using Newton's method

$$f(x) = x^2 - 3x + 2 = 0$$

- Prior to its programming implementation, we have to compute its derivative

$$f'(x) = \frac{\partial f}{\partial x} = 2x - 3$$

Programming Implementation: exercise


```
% Filename: newton1D.m
% Demo of Newton-Raphson's method in 1D
%
% f(x) = x^2 - 3*x + 2 = 0
% f'(x) = 2*x - 3

function x = newton1D(  )

% Guess solution
x = 5;

% Tolerance
eps = 1e-6;

% Error
err = eps*10; % initialize to a value larger than eps

while err > eps
    deltaX = f(x) / df(x);
    x = x - deltaX;
    err = abs( f(x) );
end

fprintf('Solution is %f\n', x);

end

function s = f(x)
s = x^2 - 3*x + 2;
end

function s = df(x)
s = 2*x - 3;
end
```

Failure of Newton's Method

- Newton's method does not always work. However, failure analysis of this method is beyond the scope of this class. Below are a few possible failure modes.
- Division by zero when the derivative is zero.

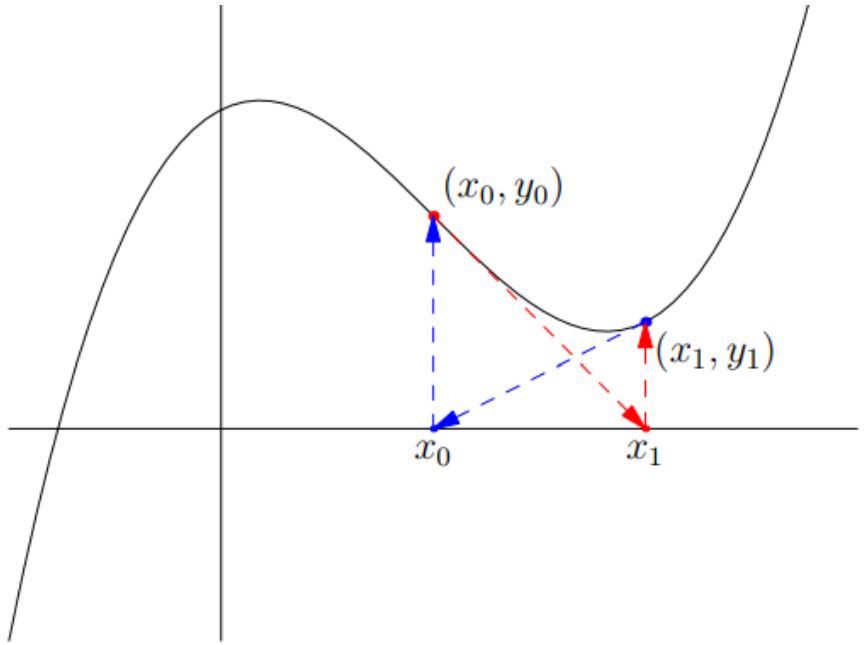
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Failure of Newton's Method

- Newton's method does not always work. However, failure analysis of this method is beyond the scope of this class. Below are a few possible failure modes.
- Division by zero when the derivative is zero.

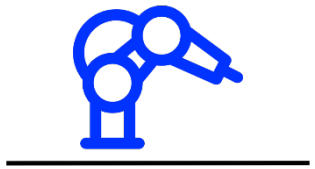
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- Cycling between two points.



Newton's method cycling between x_0 and x_1.

MIT OpenCourseWare
18.01SC Single Variable Calculus Fall 2010
https://ocw.mit.edu/courses/mathematics/18-01sc-single-variable-calculus-fall-2010/



Module 4

Newton’s method for nonlinear systems – system of equations

- Algorithm
- Programming Implementation

Newton's Method for a System of Equations

STRUCTURES-COMPUTER INTERACTION LABORATORY

Assume that we have a system of N nonlinear equations involving N variables:

$$f_1(x_1, x_2, \ldots, x_N) = 0, \tag{1.3}$$
$$f_2(x_1, x_2, \ldots, x_N) = 0, \tag{1.4}$$
$$f_N(x_1, x_2, \ldots, x_N) = 0, \tag{1.5}$$

Newton's Method for a System of Equations

Assume that we have a system of N nonlinear equations involving N variables:

$$f_1(x_1, x_2, \ldots, x_N) = 0, \tag{1.3}$$
$$f_2(x_1, x_2, \ldots, x_N) = 0, \tag{1.4}$$
$$f_N(x_1, x_2, \ldots, x_N) = 0, \tag{1.5}$$

which can be written succinctly as

$$\mathbf{f}(\mathbf{x}) = \mathbf{0}, \tag{1.6}$$

where

$$\mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ \ldots \\ f_N \end{bmatrix}, \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \ldots \\ x_N \end{bmatrix}. \tag{1.7}$$

Newton's Method for a System of Equations

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Single equation

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbb{J}(\mathbf{x})\backslash\mathbf{f}(\mathbf{x})$$

System of equations

The multivariable counterpart to the scalar derivative ($f'(x)$) is the Jacobian matrix of size $N \times N$:

$$\mathbb{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_N} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_N} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial f_N}{\partial x_1} & \frac{\partial f_N}{\partial x_2} & \cdots & \frac{\partial f_N}{\partial x_N} \end{bmatrix}.$$

$\mathbb{J}(\mathbf{x})\backslash\mathbf{f}(\mathbf{x})$ is the solution to the system of linear equations: $\mathbb{J}(\mathbf{x})\Delta\mathbf{x} = \mathbf{f}(\mathbf{x})$. We prefer to write $\mathbb{J}(\mathbf{x})\backslash\mathbf{f}(\mathbf{x})$ instead of $\mathbb{J}^{-1}(\mathbf{x})\mathbf{f}(\mathbf{x})$.

Programming Implementation

As shown below, we can formulate a simple algorithm that implements Newton's method where the functions $\mathtt{f}$ and $\mathtt{J}$ evaluate $\mathbf{f}(\mathbf{x})$ and $\mathbb{J}(\mathbf{x})$.

while $\mathtt{err} > \epsilon$

$$\Delta\mathbf{x} = \mathtt{J}(\mathbf{x}) \backslash \mathtt{f}(\mathbf{x})$$

$$\mathbf{x} = \mathbf{x} - \Delta\mathbf{x}$$

$$\mathtt{err} = \mathtt{sum}(\mathtt{abs}(\mathtt{f}(\mathbf{x})))$$

end of while

Programming Implementation: exercise

- Let us solve the following system of equations using Newton's method.

$$f_1 \equiv \quad x_1^3 + x_2 \quad = 0, \tag{1.9}$$

$$f_2 \equiv \quad -x_1 + x_2^3 + 1 \quad = 0, \tag{1.10}$$

The Jacobian is

$$\mathbb{J} = \begin{bmatrix} 3x_1^2 & 1 \\ -1 & 3x_2^2 \end{bmatrix}.$$

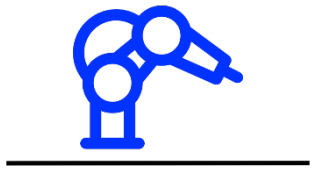
Programming Implementation: exercise

STRUCTURES-COMPUTER INTERACTION LABORATORY

```
% Filename: newton2D.m
% Demo of Newton-Raphson's method in 2D
%
% f(x) = [   x1^3+ x2   - 1;
%          -x1  + x2^3 + 1  ]
%
% J(x) = [  3x1^2,  1;
%          -1,     -3x2^2  ]
%

function x = newton2D(  )

% Guess solution
x = [0.5; 0.5];

% Tolerance
eps = 1e-6;

% Error
err = eps*10; % initialize to a value larger than eps

while err > eps
    deltaX = J(x) \ f(x);
    x = x - deltaX;
    err = abs( sum( f(x) ) );
end

fprintf('Solution is x1=%f, x2=%f\n', x(1), x(2));

end

function s = f(x)
s = [x(1)^3 + x(2) - 1; ...
    -x(1) + x(2)^3 + 1];
end

function s = J(x)
s = [   3*x(1)^2,       1;      ...
        -1,             3*x(2)^2];
end
```

Coming soon ...

- Read **Chapter 1: Newton's Method for Nonlinear Systems** of the course notes and familiarize yourself with implementing Newton's method using your preferred programming language
- Prepare yourself for the next class by reading **Chapter 2: Simulation of Mass-Spring-Damper System** of the course notes
- < Homework 1: Demo on Bending of Papers vs. Euler-Bernoulli Beam Bending >



Module 5

Mass-Spring-Damper System with Single Degree of Freedom

- Continuous Equation of Motion
- Discrete Equations of Motion

Goal

- Write a computer program to simulate a mass-spring-damper system
  - “Simulation” usually means computing the “configuration” of the system as a function of time

Resources

- Read "*Chapter 2: Simulation of mass-spring-damper system*" of Course Notes
- Wikipedia article on "Harmonic Oscillator" is also useful [https://en.wikipedia.org/wiki/Harmonic_oscillator]

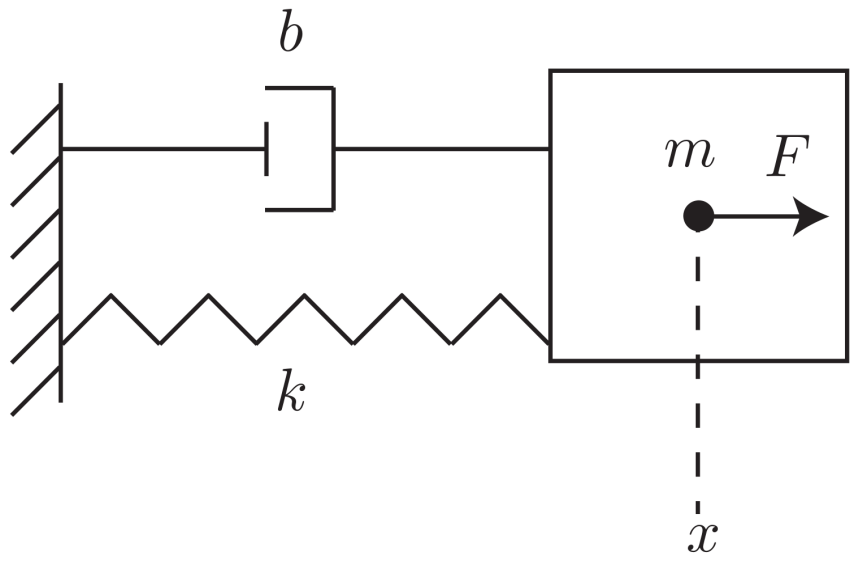
Mass-Spring-Damper System

- Single degree of freedom, x
  - This parameter alone completely describes the "configuration" of the system
  - "Simulation" = computation of the function $x(t)$

- Forces acting on the point mass:

1. spring force, $-kx$,
2. damping force, $-b\dot{x}$, and
3. the external force, $F = F_0 \sin(\omega t)$.

$$\dot{(\,)} = \frac{\partial}{\partial t}(\,)$$



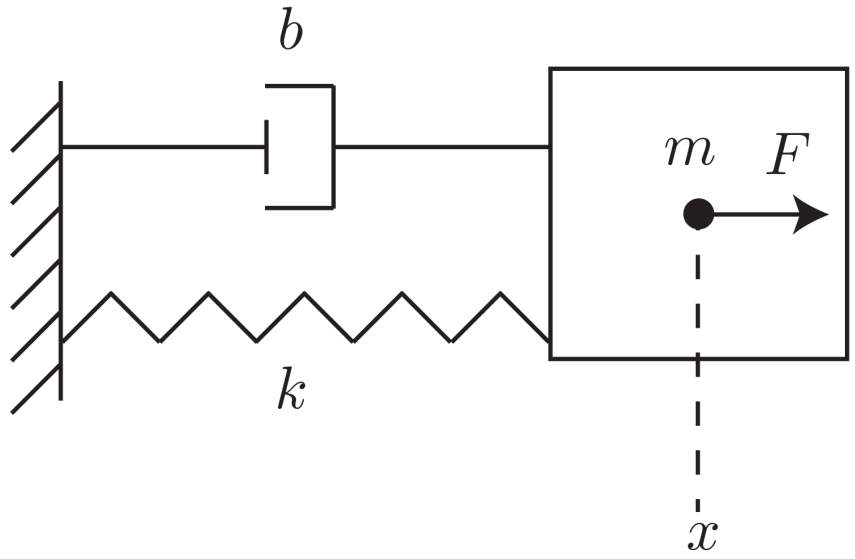
k is spring stiffness [unit: N/m]
b is viscous damping coefficient [unit: N-s/m]
F_0 is driving amplitude [unit: N]
ω is driving frequency [unit: rad/s]

Continuous Equation of Motion (EOM)

- Newton's second law

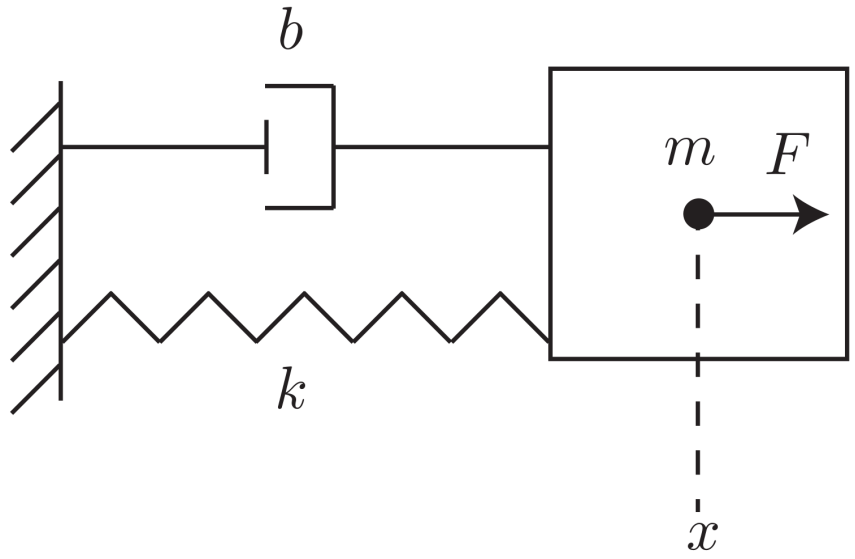
$$m\ddot{x} = -kx - b\dot{x} + F_0 \sin(\omega t)$$

$$\Longrightarrow \boxed{m\ddot{x} + kx + b\dot{x} - F_0 \sin(\omega t) = 0}$$



Discrete Equations of Motion (EOM)

- "Discrete" representation is necessary for computer simulation
- We divide "continuous" time *t* into small segments of time:



$$\text{Time array} = [0,\ \Delta t,\ 2\Delta t, \ldots, t_k,\ t_{k+1}, \ldots, \text{maxTime}],$$
$$\text{where } t_{k+1} = t_k + \Delta t.$$

- Simulation scheme: position and velocity at t_k are known. We have to compute the position and velocity at t_{k+1} by solving the equation of motion.

$$[x(t_{k+1}), \dot{x}(t_{k+1})] = \text{massSpringDamper}(\ x(t_k), \dot{x}(t_k)\ )$$

Unknown Known

Discrete Equations of Motion (EOM)

- Task at hand:

Convert this equation

$$m\ddot{x} + kx + b\dot{x} - F_0 \sin(\omega t) = 0$$

to a format involving $[x(t_{k+1}), \dot{x}(t_{k+1})]$ (unknown) and $[x(t_k), \dot{x}(t_k)]$ (known).

$$\dot{x} \rightarrow \frac{x(t_{k+1}) - x(t_k)}{\Delta t}$$

$$\ddot{x} \rightarrow \frac{1}{\Delta t}\left[\frac{x(t_{k+1}) - x(t_k)}{\Delta t} - \dot{x}(t_k)\right]$$

Continuous

$$m\ddot{x} + kx + b\dot{x} - F_0 \sin(\omega t) = 0$$

Discrete

$$\rightarrow \frac{m}{\Delta t}\left[\frac{x(t_{k+1}) - x(t_k)}{\Delta t} - \dot{x}(t_k)\right] + kx(t_{k+1}) + b\frac{x(t_{k+1}) - x(t_k)}{\Delta t} - F_0 \sin(\omega\, t_{k+1}) = 0$$

$$\dot{x}(t_{k+1}) = \frac{x(t_{k+1}) - x(t_k)}{\Delta t}$$

Discrete Equations of Motion (EOM)

- Plan:
  1. Use the first equation to solve for position $x(t_{k+1})$ [Newton's Method]

$$\frac{m}{\Delta t}\left[\frac{x(t_{k+1})-x(t_k)}{\Delta t}-\dot{x}(t_k)\right]+kx(t_{k+1})+b\frac{x(t_{k+1})-x(t_k)}{\Delta t}-F_0\sin(\omega\, t_{k+1})=0$$

  2. Use the second equation to solve for velocity $\dot{x}(t_{k+1})$ [Trivial]

$$\dot{x}(t_{k+1})=\frac{x(t_{k+1})-x(t_k)}{\Delta t}$$

Newton's method to solve EOM

$$f(x(t_{k+1})) \equiv \frac{m}{\Delta t}\left[\frac{x(t_{k+1})-x(t_k)}{\Delta t}-\dot{x}(t_k)\right]+kx(t_{k+1})+b\frac{x(t_{k+1})-x(t_k)}{\Delta t}-F_0\sin(\omega\, t_{k+1})$$

$$f'(x(t_{k+1})) = \frac{\partial f(x(t_{k+1}))}{\partial x(t_{k+1})} = \frac{m}{\Delta t^2}+k+\frac{b}{\Delta t}$$

Programming Implementation

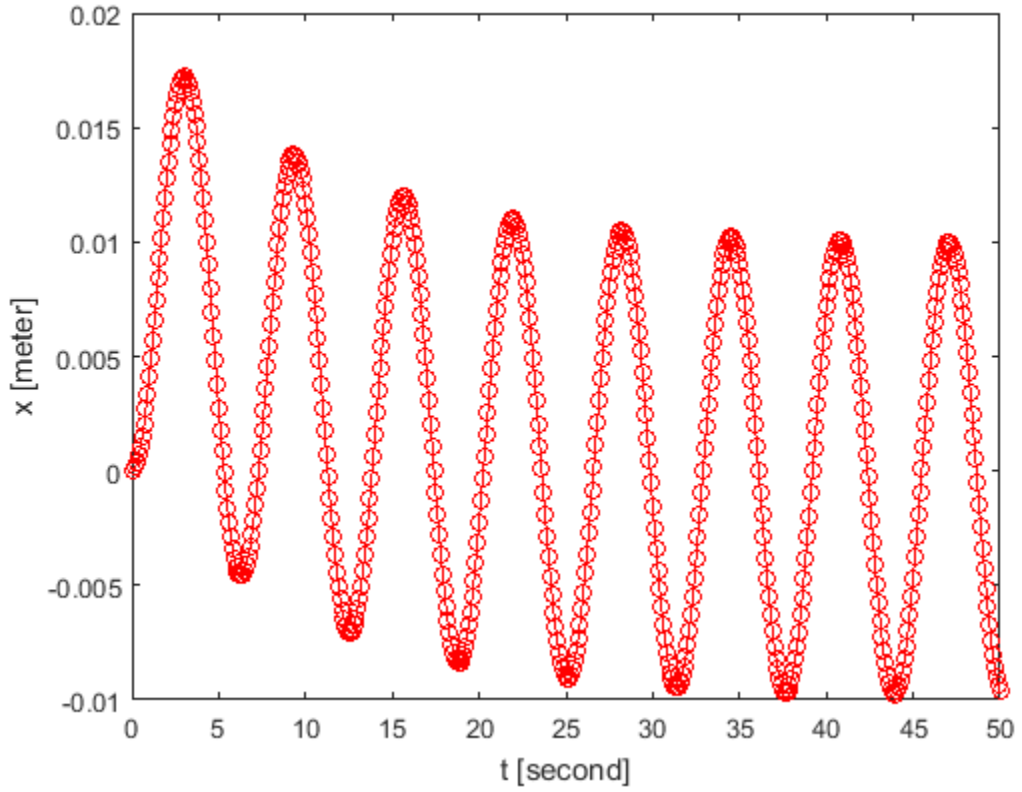
- Physical parameters:
  - Mass, $m = 1\ kg$
  - Spring constant, k=$1\ N/m$
  - Damping coefficient, $b = 10\ Ns/m$
  - External force magnitude, F_0=$0.1\ N$
  - External force frequency, ω=$1\ Hz$
- Initial conditions: the system is initially at rest. $x(0) = 0, \dot{x}(0) = 0.$

Programming Implementation


```
% Filename: massSpringDamper.m
% Simulation of a mass-spring-damper system
function massSpringDamper()
global m K b F0 omega dt

m = 1; % mass
K = 1; % spring constant
b = 10; % viscous damping coefficient
F0 = 0.1; % driving amplitude
omega = 1; % driving frequency
maxTime = 50; % total time of simulation
dt = 1e-1; % discrete time step
eps = 1e-6 * F0; % error tolerance

t = 0:dt:maxTime; % time
x = zeros(size(t)); % position
u = zeros(size(t)); % velocity

% Initial condition: position and velocity are zero
x(1) = 0;
u(1) = 0;

for k=1:length(t)-1 % march over time steps
    x_old = x(k);
    u_old = u(k);
    t_new = t(k+1);

    x_new = x_old; % guess solution
    err = eps * 100; % initialize to a large value
    while err > eps
        deltaX = f(x_new, x_old, u_old, t_new) / ...
            df(x_new, x_old, u_old, t_new);
        x_new = x_new - deltaX;
        err = abs( f(x_new, x_old, u_old, t_new) );
    end

    u_new = (x_new - x_old) / dt;

    x(k+1) = x_new; % store solution
    u(k+1) = u_new;
end

% Plot it
plot(t, x, 'ro-'); xlabel('t [second]'); ylabel('x [meter]');
end

function s = f(x_new, x_old, u_old, t_new)
global m K b F0 omega dt
s = m/dt * ( (x_new-x_old)/dt - u_old ) + K*x_new + b*(x_new-x_old)/dt ...
    - F0*sin(omega*t_new);
end

function s = df(x_new, x_old, u_old, t_new)
global m K b dt
s = m/dt^2 + K + b/dt;
end
```

- Code is available on the course website

Programming Implementation



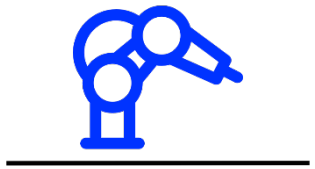
Assignment (ungraded)

- What happens if you supply inaccurate derivative (Jacobian) to the simulation?
  - Intentionally change the derivative to introduce an error, e.g.

$$f'(x(t_{k+1})) = \frac{\partial f(x(t_{k+1}))}{\partial x(t_{k+1})} = \frac{m}{\Delta t^2} + k + \frac{b}{\Delta t}$$

$$f'(x(t_{k+1})) = \frac{\partial f(x(t_{k+1}))}{\partial x(t_{k+1})} = 0.5 \times \frac{m}{\Delta t^2} + k + \frac{b}{\Delta t}$$

  - What difference does it make to your simulation result?



Module 6

Implicit vs Explicit Methods

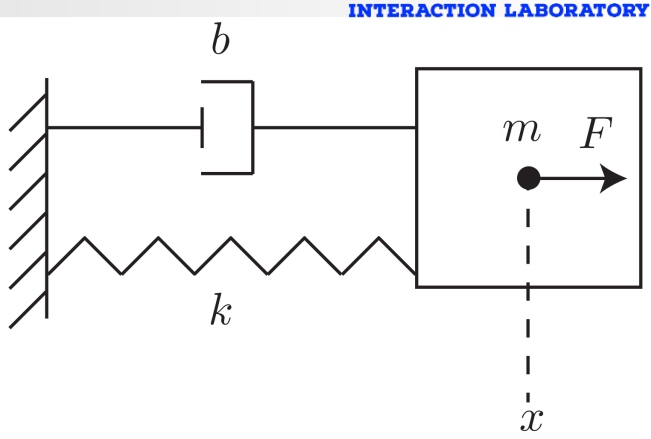
- Why do we prefer an implicit approach?

Goal

- Understand the difference between implicit and explicit methods

Implicit Method

$$\frac{m}{\Delta t}\left[\frac{x(t_{k+1})-x(t_k)}{\Delta t}-\dot{x}(t_k)\right]+kx(t_{k+1})+b\frac{x(t_{k+1})-x(t_k)}{\Delta t}-F_0\sin(\omega\, t_{k+1})=0$$



Newton's method:

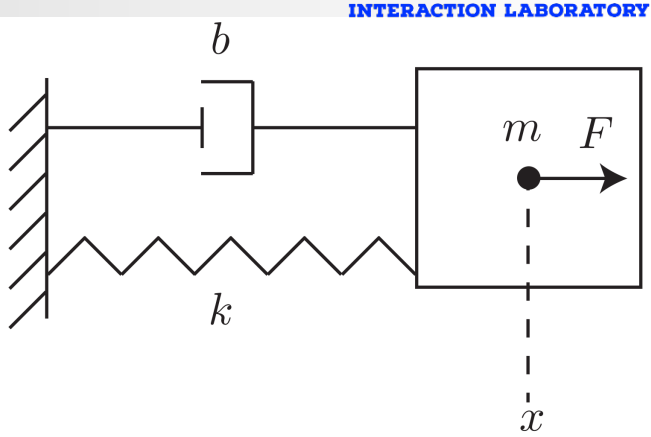
$$f(x(t_{k+1}))\equiv\frac{m}{\Delta t}\left[\frac{x(t_{k+1})-x(t_k)}{\Delta t}-\dot{x}(t_k)\right]+kx(t_{k+1})+b\frac{x(t_{k+1})-x(t_k)}{\Delta t}-F_0\sin(\omega\, t_{k+1})=0$$

$$f'(x(t_{k+1}))=\frac{\partial f(x(t_{k+1}))}{\partial x(t_{k+1})}=\frac{m}{\Delta t^2}+k+\frac{b}{\Delta t}$$

Explicit Method

Implicit

$$\frac{m}{\Delta t}\left[\frac{x(t_{k+1})-x(t_k)}{\Delta t}-\dot{x}(t_k)\right]+kx(t_{k+1})+b\frac{x(t_{k+1})-x(t_k)}{\Delta t}-F_0\sin(\omega t_{k+1})=0$$



Information from the next time step t_{k+1}

Explicit

$$\frac{m}{\Delta t}\left[\frac{x(t_{k+1})-x(t_k)}{\Delta t}-\dot{x}(t_k)\right]+kx(t_k)+b\dot{x}(t_k)-F_0\sin(\omega t_k)=0$$

Information from the old time step t_k (Known)

$$\Longrightarrow\ x(t_{k+1})=x(t_k)+\Delta t\left[\dot{x}(t_k)-\frac{\Delta t}{m}\left(kx(t_k)+b\dot{x}(t_k)-F_0\sin(\omega\, t_k)\right)\right]$$

Explicit Method

- Simple algebraic manipulation directly gives us the solution
- Newton's method is not necessary

Why Implicit Method?

- Explicit method is easier and simpler. Why do we choose implicit method?

- A key metric in numerical simulation is the time step size (Δt)
  - Smaller time step size leads to more accurate results
  - BUT smaller time step size requires more computation time

- Implicit method allows larger time step size (Δt) and results in faster computation time

Module 7

Mass-Spring-Damper System with Multiple Degrees of Freedom

- Continuous Equation of Motion
- Discrete Equations of Motion

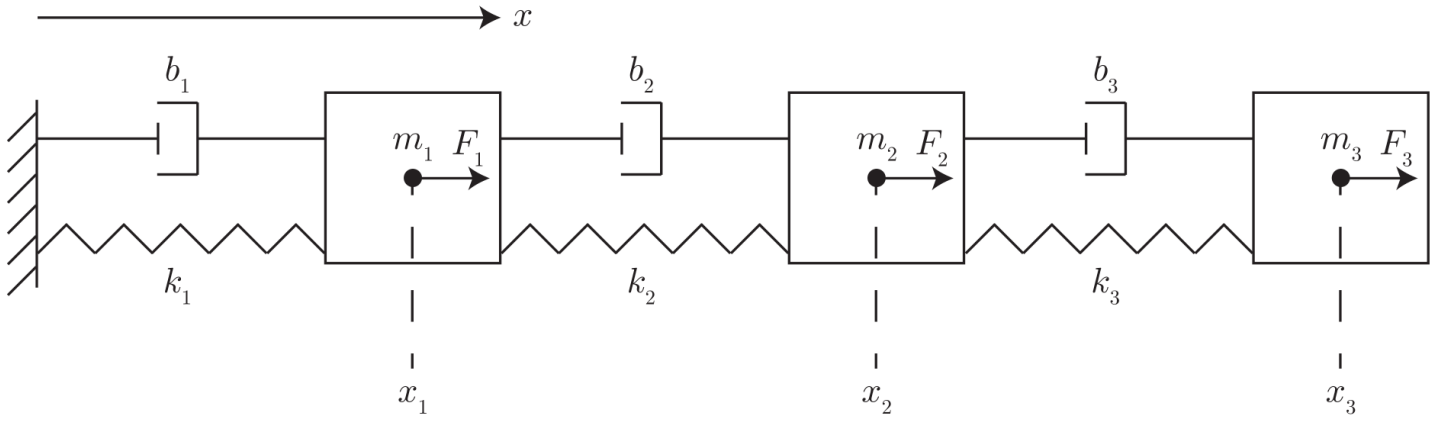
Goal

- Write a computer program to simulate a system comprised of multiple masses, springs, and dampers

Resources

- Read "*Chapter 3: Simulation of multi-mass spring damper system*" of Course Notes
- We will use Newton's method to solve a system of equations; read *"Chapter 1: Newton's method for nonlinear systems"*

Mass-Spring-Damper System

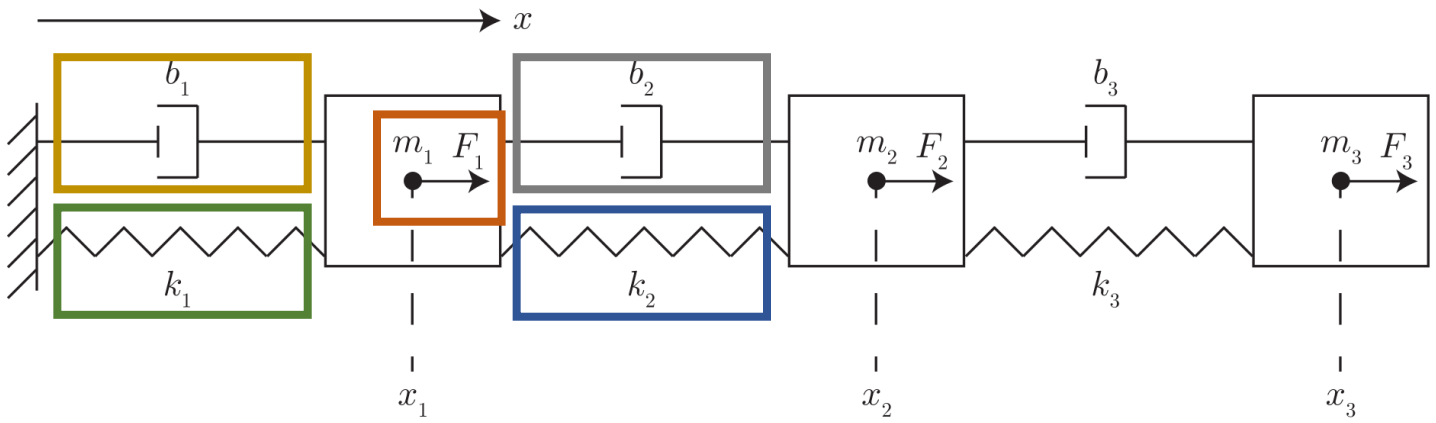


k_i is spring stiffness [unit: N/m]
bi is viscous damping coefficient [unit: N-s/m]
${F_0}^i$ is driving amplitude [unit: N]
ω_i is driving frequency [unit: rad/s]

- Multiple degrees of freedom, $\boldsymbol{x} = [x_1, x_2, x_3]^T$
  - Superscript $(\ )^T$ denotes transposition operator

- These three parameters completely describe the system

Continuous Equation of Motion (EOM)



Let us tabulate the forces on the first mass m_1:

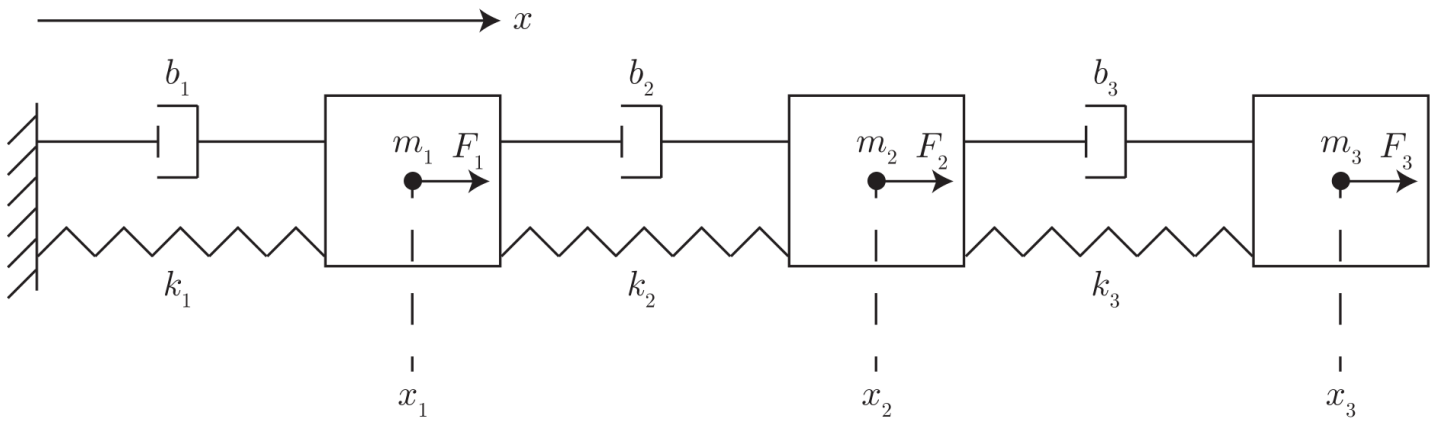
1. spring forces, $-k_1 x_1$ (left spring) and $k_2 (x_2 - x_1)$ (right spring),
2. damping forces, $-b_1 \dot{x}_1$ (left damper) and $b_2 (\dot{x}_2 - \dot{x}_1)$ (right damper), and
3. the external force, $F_1 = F_0^1 \sin(\omega_1 t)$

Continuous Equation of Motion for x_1:

$$m_1 \ddot{x}_1 = -k_1 x_1 + k_2 (x_2 - x_1) - b_1 \dot{x}_1 + b_2 (\dot{x}_2 - \dot{x}_1) + F_0^1 \sin(\omega_1 t)$$

$$m_1 \ddot{x}_1 + (k_1 + k_2) x_1 - k_2 x_2 + (b_1 + b_2) \dot{x}_1 - b_2 \dot{x}_2 - F_0^1 \sin(\omega_1 t) = 0$$

Continuous Equations of Motion (EOM)



Continuous Equation of Motion for x_2:

$$m_2\ddot{x}_2 + (k_2 + k_3)x_2 - k_2x_1 - k_3x_3 + (b_2 + b_3)\dot{x}_2 - b_2\dot{x}_1 - b_3\dot{x}_3 - F_0^2 \sin(\omega_2 t) = 0$$

General Case: Continuous Equation of Motion for x_i in N DOF system:

$$\begin{aligned} m_i\ddot{x}_i + &(k_i + k_{i+1})x_i - k_ix_{i-1} - k_{i+1}x_{i+1} \\ &+ (b_i + b_{i+1})\dot{x}_i - b_i\dot{x}_{i-1} - b_{i+1}\dot{x}_{i+1} - F_0^i \sin(\omega_i t) = 0 \end{aligned}$$

Discrete Equations of Motion (EOM)

$$m_i\ddot{x}_i+(k_i+k_{i+1})x_i-k_ix_{i-1}-k_{i+1}x_{i+1} + (b_i+b_{i+1})\dot{x}_i-b_i\dot{x}_{i-1}-b_{i+1}\dot{x}_{i+1}-F_0^i\sin(\omega_i t)=0$$

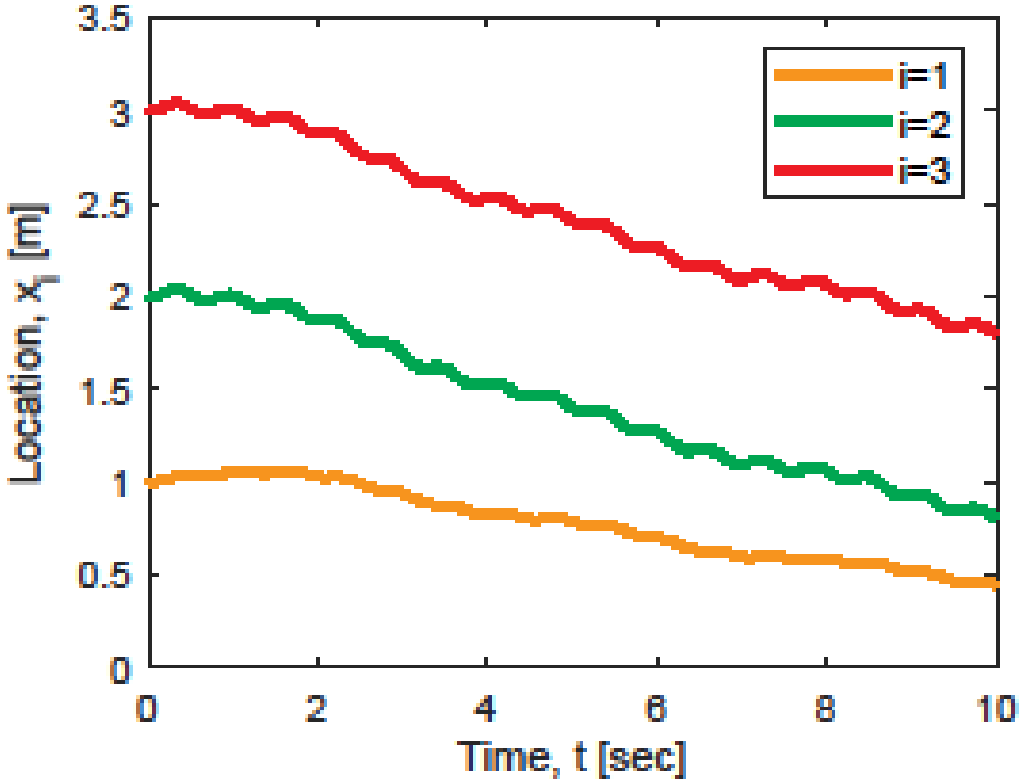
$$\begin{aligned} f_i \equiv & \frac{m_i}{\Delta t}\left[\frac{x_i(t_{k+1})-x_i(t_k)}{\Delta t}-\dot{x}_i(t_k)\right]+ \\ & (k_i+k_{i+1})\,x_i(t_{k+1})-k_ix_{i-1}-k_{i+1}x_{i+1}+ \\ & (b_i+b_{i+1})\,\frac{x_i(t_{k+1})-x_i(t_k)}{\Delta t}-b_i\frac{x_{i-1}(t_{k+1})-x_{i-1}(t_k)}{\Delta t}-b_{i+1}\frac{x_{i+1}(t_{k+1})-x_{i+1}(t_k)}{\Delta t} \\ & -F_0^i\sin(\omega_i\,t_{k+1})=0 \end{aligned}$$

Newton's method for a system of equations requires the **Jacobian** associated with the system of equations:

$$\mathbb{J}_{ij}=\frac{\partial f_i}{\partial x_j}=\begin{cases} \frac{m_i}{\Delta t^2}+(k_i+k_{i+1})+\frac{b_i+b_{i+1}}{\Delta t}, & \text{for } j=i \\ -k_i-\frac{b_i}{\Delta t}, & \text{for } j=i-1 \\ -k_{i+1}-\frac{b_{i+1}}{\Delta t}, & \text{for } j=i+1 \\ 0, & \text{otherwise.} \end{cases}$$

Programming Implementation

- $m_1 = 10$ kg, $m_2 = 10^{-2}$ kg, and $m_3 = 10^{-3}$ kg,
- $k_1 = 1$ N/m, $k_2 = 5$ N/m, and $k_3 = 0.1$ N/m,
- $b_1 = 10$ Ns/m, $b_2 = 50$ Ns/m, and $b_3 = 100$ Ns/m,
- $F_0^1 = 0.1$ N, $F_0^2 = 10$ N, and $F_0^3 = 1$ N,
- $\omega_1 = 0.1$ rad/s, $\omega_2 = 10$ rad/s, and $\omega_3 = 2$ rad/s, and
- $x_{10} = 1$ m, $x_{20} = 2$ m, and $x_{30} = 3$ m.



- A MATLAB code (massSpringDamper_multi.m) that implements the example below is available to the students.

Assignment (ungraded)

- Implement the simulation of multiple mass-spring-dampers yourself, without consulting the code made available to you